

บทที่ 4

การพัฒนาแอปพลิเคชันสำหรับอุปกรณ์ไร้สายด้วยจาวา

4.1 คอมโพเนนต์หลักของส่วนติดต่อกับผู้ใช้

เจทูเอ็มอี สามารถทำงานบนอุปกรณ์ที่หลากหลาย ตั้งแต่สมาร์ตการ์ดและโทรศัพท์เคลื่อนที่ ไปจนถึงอุปกรณ์ set-top-box สำหรับโทรทัศน์และเครื่องใช้ไฟฟ้าในบ้าน อุปกรณ์เหล่านี้มีความสามารถในการแสดงผลและทรัพยากรที่แตกต่างกันค่อนข้างมาก บริษัทซัมซุง ไมโครซิสเต็มส์ตระหนักถึงปัญหานี้จึงไม่ได้กำหนดคลาสส่วนติดต่อกับผู้ใช้ซึ่งสมบูรณ์แบบใน เจทูเอ็มอี หากแต่กำหนด API ของส่วนติดต่อกับผู้ใช้ในลักษณะโพรไฟล์ของอุปกรณ์เคลื่อนที่ (Mobile Information Device Profile : MIDP) หรือถ้าเป็น API ส่วนติดต่อกับผู้ใช้สำหรับพีดีเอ ก็จะระบุไว้ในโพรไฟล์ของพีดีเอ (Personal Digital Assistant Profile : PDAP) เป็นต้น

API ของส่วนติดต่อกับผู้ใช้ในเจทูเอสอีกำหนดไว้ในชุดเครื่องมือ Abstract Window Toolkit (AWT) ซึ่งออกแบบสำหรับเครื่องคอมพิวเตอร์ตั้งโต๊ะที่มีอุปกรณ์การชี้ (Pointer Device) จอภาพขนาดใหญ่พร้อมหน่วยความจำที่พอเพียง ขณะที่หน้าจอแสดงผลไร้สายมีขนาดเล็กกว่า และใช้แบนด์วิดท์เพื่อป้อนข้อมูลเข้าแทนที่จะเป็นคีย์ ดังนั้นกลุ่มผู้เชี่ยวชาญการออกแบบ MIDP จึงได้กำหนด API ของส่วนติดต่อกับผู้ใช้ขึ้นมาใหม่ทั้งหมดโดยไม่ได้อ้างอิงซึบเซตของ AWT เลยซึ่ง API ของส่วนติดต่อกับผู้ใช้เหล่านี้กำหนดไว้ในแพ็คเกจ javax.microedition.lcdui

API ของส่วนติดต่อกับผู้ใช้ใน MIDP มี 2 กลุ่มหลักๆ ด้วยกันคือ API ระดับสูงและระดับล่าง API ระดับสูงออกแบบให้เป็น Abstract เน้นความยืดหยุ่นในการโยกย้ายไปใช้งานในระบบอื่น แอปพลิเคชันที่ใช้ API ระดับสูงจะควบคุมวิธีแสดงผลได้น้อยมากและเข้าถึงได้เฉพาะอ็อบเจกต์ระดับสูงเท่านั้น ในทางตรงกันข้ามการใช้ API ระดับล่างจะสามารถควบคุมการแสดงผลได้เต็มที่ สามารถเข้าถึงอุปกรณ์ป้อนข้อมูล และจัดการอ็อบเจกต์พื้นฐานที่เกิดจากการตอบโต้ของผู้ใช้ได้ อย่างไรก็ตามโปรแกรมที่ใช้ API ระดับล่างนี้จะขาดความยืดหยุ่น ยึดติดกับอุปกรณ์มากเกินไป โดยจะต้องพิจารณาถึงข้อดีข้อเสียในการออกแบบโปรแกรมด้วย

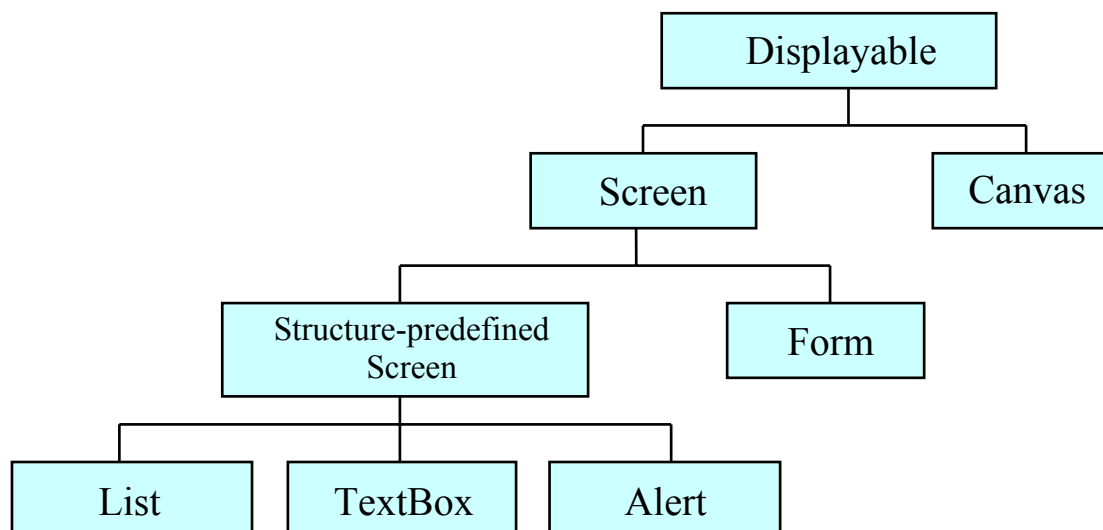
4.1.1 Displayable และ Display ฟังก์ชันหลักอย่างหนึ่งของส่วนติดต่อกับผู้ใช้ ก็คือ การแสดงผลข้อมูลบนหน้าจอใน J2ME MIDP ออบเจกต์ Displayable จะเก็บข้อมูลที่ต้องการนำเสนอ และออบเจกต์ Display จะนำออบเจกต์ Displayable นั้นไปแสดงบนจอให้ผู้ใช้เห็น

4.1.1.1 Displayable เป็นคลาสหลักของส่วนติดต่อกับผู้ใช้ของ MIDP สามารถเก็บภาพกราฟิกที่สร้างเฉพาะอุปกรณ์และนำไปแสดงบนจอ

MIDP กำหนด API ของส่วนติดต่อกับผู้ใช้ออกเป็น 2 กลุ่มคือ API ระดับสูงและระดับล่าง คลาส Screen เป็นคลาสย่อยของ Displayable ที่ใช้ API ระดับสูง ส่วนคลาส Canvas เป็นคลาสย่อยของ Displayable ที่ใช้ API ระดับล่าง

ขนาดและรูปร่างของหน้าจออุปกรณ์จะแตกต่างกันออกไป เนื่องจาก API ระดับสูงจะจัดการกับความแตกต่างของขนาดหน้าจอ ดังนั้น ในการเขียนแอปพลิเคชันจึงไม่ต้องพะวงเรื่องความแตกต่างของขนาดหน้าจอมากนัก แต่หากเป็นแอปพลิเคชันที่ใช้ API ระดับล่างจะต้องระวังเรื่องความแตกต่างของหน้าจอแสดงผลเหล่านี้

Displayable ทุกตัวต้องรีจิสเตอร์ Command และ CommandListener เพื่อที่จะสามารถรับอีเวนต์ของคอมมานด์ได้



รูปที่ 4.1 โครงสร้างลำดับชั้นของคลาส Displayable

4.1.1.1.1 Screen เป็นซับคลาสของ Displayable ซึ่งนำ API ระดับสูงมาใช้ ประกอบด้วยซูเปอร์คลาส Alert , Form , List และ TextBox อาจมีชื่อหรือแถบตัวอักษรวิ่ง (ticker tape) กำกับหรือจะไม่มีก็ได้

Screen เป็นคลาสแบบ Abstract เมธอดซึ่งกำหนดวิธีการแสดงผลและการโต้ตอบกับเนื้อหาในออบเจกต์ก็เป็นเมธอดแบบ Abstract ด้วย นำมาใช้โดยซับคลาสของ Screen มีเพียงเมธอดสำหรับปรับปรุงหน้าจอเวลาเมื่อต้องการแสดงผลเท่านั้นที่อยู่ในคลาส Screen อย่างไรก็ดี ไม่สามารถเพิ่มเติมอะไรในคลาส Screen ได้เอง เนื่องจากใช้เมธอดแบบ Abstract ซึ่งไม่สามารถเข้าถึงได้จากภายนอกแพ็คเกจ javax.microedition.lcdui

4.1.1.1.2 Canvas เป็นซับคลาสของ Displayable ซึ่งใช้ API ระดับล่าง แอปพลิเคชันสามารถควบคุมได้ว่าแสดงผลอะไรและอย่างไรบ้าง และยังสามารถเข้าถึงอีเวนต์ระดับล่างเช่น การกดปุ่ม ได้ด้วย Canvas เป็นคลาส Abstract แอปพลิเคชันต้องสร้างซับคลาสร้อยเพื่อใช้งาน

4.1.1.2 Display เป็นตัวจัดการการแสดงผล และใช้เมธอดเหล่านี้สำหรับอ่านค่าพารามิเตอร์ของอุปกรณ์และนำออบเจกต์ไปแสดงผลบนหน้าจอ

ในโปรแกรม MIDlet หนึ่งๆ จะมีออบเจกต์ของ Display ได้เพียงตัวเดียวเท่านั้น แอปพลิเคชันสามารถอ้างถึงออบเจกต์ Display ได้ด้วยเมธอด getDisplay() และสามารถเรียกเมธอดนี้ได้ทุก

เมื่อตั้งแต่เริ่มเรียกใช้ MIDlet.startApp() ไปจนถึง MIDlet.destroyApp() ออบเจ็กต์ Display ทั้งหมดที่ได้จาก getDisplay() จะยังคงอยู่ตลอด จนกว่าจะเสร็จสิ้นโปรแกรม MIDlet

Displayable ปัจจุบัน แอปพลิเคชันจะมีออบเจ็กต์ Displayable ที่จะแสดงผลบนอุปกรณ์หรือมีการทำงานโต้ตอบกับผู้ใช้ได้เพียงออบเจ็กต์เดียวเท่านั้น โดยเรียก Displayable นี้ว่า Displayable ปัจจุบัน ผู้ใช้จะป้อนข้อมูลหรือโต้ตอบได้เฉพาะกับ Displayable ปัจจุบันเท่านั้น คลาส Displayable มีเมธอด getCurrent() สำหรับดึง Displayable ปัจจุบัน และมีเมธอดไว้สำหรับแสดงผลคือ setCurrent(Displayable nextDisplayable)

แอปพลิเคชัน สามารถควบคุม Displayable ปัจจุบัน และ เรียก เมธอด Display.setCurrent() ได้ตลอดเวลา เรดของ MIDlet สามารถกำหนดหรือดึงค่า Displayable ปัจจุบันด้วย เมธอด Display.setCurrent() และ Display.getCurrent()

แอปพลิเคชัน หน้าฉากและหลังฉาก แม้สามารถสั่งให้แอปพลิเคชันหลายตัวทำงานพร้อมกันบนอุปกรณ์หนึ่งๆ อุปกรณ์ก็จะแสดงผลได้เฉพาะ Displayable ปัจจุบันเท่านั้น ดังนั้นแอปพลิเคชันใดที่มี Displayable ปัจจุบันแสดงผลอยู่บนจอจะเรียกว่าเป็นแอปพลิเคชันหน้าฉาก (foreground) ซึ่งสามารถรับอีเวนต์การป้อนข้อมูลเข้าจากผู้ใช้ได้ แอปพลิเคชันที่เหลือซึ่งไม่ได้แสดงผลจะเรียกว่าแอปพลิเคชันหลังฉาก (background)

MIDlet สามารถวางตัวเองให้อยู่หลังฉากได้โดยใช้ Display.setCurrent(null) แต่การใช้เมธอดนี้ไม่ได้กำหนด Displayable ปัจจุบันให้เป็น null จริงๆ สมมติว่า Displayable ปัจจุบันคือ cD หลังจากแอปพลิเคชันเรียก Display.setCurrent(null) แล้ว การเรียกเมธอด getCurrent() ก็ยังได้ค่าเป็น cD แอปพลิเคชัน จะกลับคืนสู่หน้าฉากโดยใช้ Display.setCurrent(Display.getCurrent())

แม้ว่า Displayable ปัจจุบันของแอปพลิเคชันจะไม่ได้วาดอะไรบนจอก็ตาม แอปพลิเคชันก็ยังรับรู้ว่ามี Displayable ใดเป็น Displayable ปัจจุบัน ซึ่งสำคัญมากแม้แต่กับแอปพลิเคชันหลังฉาก เนื่องจากเมื่อแอปพลิเคชันหลังฉากกลายเป็นแอปพลิเคชันหน้าฉาก ก็จะสามารถแสดงผล Displayable ที่ถูกต้องได้

โปรแกรม MIDlet แต่ละตัวจะมี Displayable ปัจจุบันของตัวเอง เมธอด getCurrent() จะให้ Displayable ปัจจุบันโดยใช้เมธอด setCurrent() ก็ไม่มีผลอะไรกับ Displayable ปัจจุบันของ MIDlet อื่นๆ

System Screen โดยปรกติแล้ว จะสามารถมองเห็นการแสดงผลบนหน้าจอปัจจุบันของ MIDlet ที่ทำงานหน้าฉากได้ อย่างไรก็ตามในบางสถานการณ์ระบบอาจจะสร้างหน้าจอชั่วคราวขึ้นมาบังหน้าจอปัจจุบันของแอปพลิเคชันเอาไว้ คือ หน้าจอของระบบ(system screen) โดยจะแสดงขึ้นมาเมื่อระบบต้องการแสดงเมนูเพิ่มเติม นอกเหนือไปจากการแก้ไขไฟล์ข้อความต่างๆใน Form

แม้ว่าหน้าจอของระบบ จะโผล่ขึ้นมาบังหน้าจอของแอปพลิเคชัน Displayable ปัจจุบัน ก็ยังเป็นอันเดิมไม่เปลี่ยนแปลง การเรียกใช้เมธอด getCurrent() ในขณะที่มองเห็นหน้าจอของระบบ ก็ยังได้ค่า Displayable ปัจจุบันของแอปพลิเคชัน และไม่ใช่นำหน้าจอของระบบ

ถ้าจะดูว่า Displayable ปัจจุบัน ถูกบังด้วยหน้าจอของระบบหรือไม่ สามารถตรวจสอบ ได้จาก `currentDisplayable.isShown()`

4.1.2 Image คลาส Image ใช้เก็บข้อมูลรูปภาพ ออบเจกต์ Image จะอยู่ในหน่วยความจำที่แยกจากส่วนแสดงผล ออบเจกต์ Image สามารถวางลงบน Canvas หรือวางใน Form , Alert , List หรือ ChoiceGroup ก็ได้

Image อาจเป็นรูปภาพที่แก้ไขได้ หรือแก้ไขไม่ได้ ขึ้นอยู่กับวิธีการสร้าง

4.1.2.1 รูปที่แก้ไขไม่ได้ มักมีที่มาจากกราฟิกจากไฟล์ จากแหล่งอื่นๆ ที่ใส่มาในแพ็คเกจ หรือจากเครือข่าย เมื่อสร้างรูปเสร็จแล้ว จะไม่สามารถแก้ไขได้อีก รูปที่ใส่ใน Alert , Choice หรือ Item จะต้องเป็นรูปที่แก้ไขไม่ได้เท่านั้น

4.1.2.2 รูปที่แก้ไขได้ จะสร้างในหน่วยความจำที่อยู่นอกจอ เสมือนกับว่าเป็นหน้าจอหรือ Canvas อีกอันหนึ่ง แอปพลิเคชันสามารถวาดรูปที่แก้ไขนี้ได้จากที่สร้างขึ้นมาแล้ว ด้วยเมธอด `Image.createImage(int width,int height)` โดยรูปที่แก้ไขได้จะใช้ในเทคนิคแบบบัพเฟอร์ 2 ชั้น

พารามิเตอร์ของรูป ออบเจกต์ของ image จะมีพารามิเตอร์ 3 แบบด้วยกัน คือ height width และ isMutable แอปพลิเคชันสามารถอ่านค่าพารามิเตอร์ได้โดยใช้เมธอด `getHeight()` `getWidth()` และ `isMutable()` ตามลำดับ ถ้ารูปนั้นแก้ไขได้ แอปพลิเคชันจะสามารถสร้างออบเจกต์ Graphics เพื่อสร้างรูปโดยใช้ `getGraphics()`

4.1.3 อีเวนต์และการจัดการอีเวนต์ ส่วนติดต่อกับผู้ใช้ประกอบด้วย 2 ส่วนหลักคือ หน้าจอที่ทำหน้าที่แสดงผล และตอบสนองการโต้ตอบของผู้ใช้ ส่วนติดต่อกับผู้ใช้ในอีเวนต์จะทำหน้าที่ประมวลผลอีเวนต์ที่เกิดขึ้น อีเวนต์และตัวจัดการอีเวนต์มีทั้งระดับสูงและระดับล่างเช่นเดียวกันกับ API ของ Displayable

4.2 การใช้ API ระดับสูงในการพัฒนาส่วนติดต่อกับผู้ใช้

API ระดับสูงเน้นความสามารถในการโยกย้ายไปใช้กับอุปกรณ์อื่นๆ ชั้นคลาสของ Screen มี 2 กลุ่ม กลุ่มแรก ได้แก่ List TextBox และ Alert ซึ่งมีโครงสร้างที่กำหนดไว้ล่วงหน้าแล้ว กับกลุ่มที่สอง ได้แก่คลาส Form ซึ่งกำหนดโครงสร้างโดยแอปพลิเคชัน

4.2.1 List และ Choice เมื่อเริ่มการทำงาน แอปพลิเคชันมักจะแสดงเมนูซึ่งประกอบด้วยฟังก์ชันต่างๆ ให้คุณได้เลือกตามต้องการ List และ ChoiceGroup ต่างแสดงรายการทางเลือก โดยทั้งคู่ใช้อินเตอร์เฟซ Choice ที่กำหนดไว้ในแพ็คเกจ `javax.microedition.lcdui`

4.2.1.1 List เป็น Screen ซึ่งมีโครงสร้างที่กำหนดไว้แล้วโดยใช้อินเตอร์เฟซ Choice API ของแพ็คเกจ `javax.microedition.lcdui` กำหนด constructor ของ list ไว้ 2 แบบ โดยตัวแรกจะสร้าง List เปล่าๆขึ้นมา ซึ่งสามารถเพิ่มรายการของ choice เข้าไปได้ทีหลัง constructor ตัวที่สองจะสร้าง List ที่มีค่าเริ่มต้นอยู่ข้างใน อาร์เรย์ `stringElements` ต้องไม่เป็น null และสมาชิกทุกตัวในอาร์เรย์ต้องไม่เป็น null ด้วยเช่นกัน ความยาวของอาร์เรย์ `stringElements` จะกำหนดจำนวนรายการใน List อาร์เรย์

imageElements อาจมีค่าเป็น null ซึ่งหมายความว่ารายการใน List นั้นไม่มีภาพอยู่ ถ้าอาร์เรย์ imageElements ไม่เป็น null ก็จะต้องมีความยาวเท่ากับอาร์เรย์ stringElements รายการแต่ละรายการในอาร์เรย์ imageElements อาจจะเป็น null ได้ซึ่งหมายความว่าไม่มีภาพในรายการของ List ที่ตรงกัน

4.2.1.2 Choice ออบเจกต์ Choice มี 3 ประเภทได้แก่ implicit-choice (ใช้ได้กับ List เท่านั้น) exclusive-choice และ multiple-choice

4.2.1.2.1 Exclusive-choice จะต้องเลือก รายการใดรายการหนึ่ง ในแต่ละครั้ง เว้นเสียแต่จะไม่มีรายการใดๆ ให้เลือก

4.2.1.2.2 Implicit-choice เป็นกรณีพิเศษของ exclusive-choice เมื่อรายการที่อยู่ในโฟกัสถูกเลือกโดยอัตโนมัติเมื่อเริ่มต้น Command โดย choice ประเภทนี้ใช้ได้กับ List เท่านั้น

4.2.1.2.3 Multiple-choice สามารถเลือกหลายรายการพร้อมกันได้ในแต่ละครั้ง หรือจะ ไม่เลือกเลยก็ได้



รูปที่ 4.2 List แบบ

Implicit-choice



รูปที่ 4.3 List แบบ

exclusive-choice



รูปที่ 4.4 List แบบ

multiple-choice

ออบเจกต์ Choice จะแสดงรายการทางเลือกแบบต่างๆ ให้ผู้ใช้เลือก แต่ละรายการประกอบด้วยสตริงข้อความและรูปซึ่งอาจจะมีหรือไม่มีก็ได้

จำนวนรายการในออบเจกต์ Choice สามารถรู้ได้โดยใช้เมธอด size() แต่ละรายการสามารถอ้างถึงได้ตามดัชนีซึ่งเป็นตัวเลขจำนวนเต็มมีค่าตั้งแต่ 0 จนถึง size() - 1 โดยที่ 0 อ้างถึงรายการแรกและ size() - 1 อ้างถึงรายการสุดท้าย

ภาพในรายการ Choice อาจเว้นว่างได้ถ้าแอปพลิเคชันนั้นไม่ได้กำหนดไว้ หรือแม้จะกำหนดไว้ก็อาจเลือกไม่แสดงภาพเลย หากอุปกรณ์นั้นๆ ไม่สามารถแสดงผลได้ ถ้าเลือกที่จะแสดงภาพภาพจะติดอยู่กับสตริงข้อความและถือเสมือนว่าทั้งภาพและข้อความนั้นเป็นหน่วยเดียวกัน ภาพไม่สามารถเปลี่ยนได้ แอปพลิเคชันสามารถกำหนดภาพให้กับรายการ elementNum choice ได้โดย

Void set(int elementNum, String stringPart, Image imagePart) และอ่านภาพจาก elementNum choice ได้โดย Image getImage(int elementNum)

4.2.2 TextBox เป็น Screen ที่มีโครงสร้างกำหนดไว้แล้ว ผู้ใช้งานสามารถป้อนหรือแก้ไขข้อความใน TextBox ได้ การสร้างออบเจกต์ TextBox ทำได้โดยใช้ constructor ดังนี้

`TextBox(String title, String text, int maxSize, int constraints)`

ข้อความใน TextBox จะเก็บเป็นอาร์เรย์ของ char ดังนั้นจึงมีขนาด (ความจุ) ในการเก็บกลุ่มตัวอักษรไว้ในออบเจกต์อย่างจำกัด โดยสามารถกำหนดขนาดสูงสุดของ TextBox ในขณะที่สร้างออบเจกต์ หรืออาจใช้เมธอด `setMaxSize()` กำหนดหลังจากที่สร้างออบเจกต์ TextBox แล้ว การดูขนาดความจุทำได้โดยใช้เมธอด `getMaxSize()` เนื่องจากอุปกรณ์ไร้สายมักมีหน่วยความจำจำกัดขีดจำกัดสูงสุดที่แท้จริงขึ้นอยู่กับค่าสูงสุดที่ MIDP กำหนดไว้ ดังนั้นขนาดสูงสุดจริงของ TextBox จะจำกัดอยู่ภายในกรอบที่กำหนดโดย MIDP

การตรวจสอบขนาดความจุสูงสุดเกิดขึ้นเมื่อสร้างออบเจกต์ TextBox เมื่อผู้ใช้งานแก้ไขข้อความภายใน TextBox และเมื่อโปรแกรมเรียกใช้เมธอดเพื่อแก้ไขเนื้อหาของ TextBox เมื่อใดก็ตามที่ข้อความใน TextBox มีขนาดเกินกว่าที่กำหนดไว้ จะเกิด `IllegalArgumentException` ขึ้น

ข้อความใน TextBox อาจมีความยาวเกินกว่าจะแสดงผลได้หมดในครั้งเดียว ในกรณีนี้ผู้ใช้สามารถเลื่อนหน้าจอขึ้น-ลง เพื่อดูหรือแก้ไขข้อความส่วนต่างๆ การเลื่อนหน้าจอนี้ไม่ทำให้เกิดอิวেন্ট `application-visible` แต่อย่างใด

4.2.3 Alert เป็น Screen ที่มีโครงสร้างกำหนดไว้แล้ว ใช้สำหรับแสดงข้อมูล (ข้อความและภาพ) ให้ผู้ใช้และจะคอยเป็นเวลาครู่หนึ่ง (กำหนดโดย timeout) ก่อนที่จะแสดงหน้าจอถัดไป

4.2.4 Form และ Item

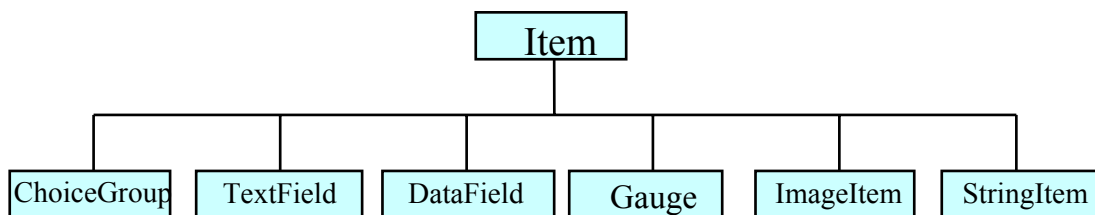
Form เป็นออบเจกต์ Screen ซึ่งสามารถบรรจุ Item ต่างๆเช่น ภาพ ข้อความ หรือ choice จำนวนเท่าใดก็ได้ การสร้างออบเจกต์ Form ทำได้ 2 แบบ คือ สร้าง Form เปล่าที่ไม่มี Item ใดๆโดยใช้ `Form(String title)` หรือสร้าง Form ที่มี Item เริ่มต้นโดยใช้ `Form(String title, Item[] item)`

สามารถดูจำนวนของ Item ใน Form ได้โดยใช้เมธอด `size()` การอ้างถึง Item ทำได้โดยใช้ดัชนีตัวเลข ซึ่งมีค่าตั้งแต่ 0 ถึง `size() - 1` อ้างถึงตัวสุดท้าย

สามารถดึงออบเจกต์ Item จาก Form ได้โดยใช้ `get(int itemNum)`

Item หนึ่งจะอยู่ได้ใน Form เดียวเท่านั้น หากแอปพลิเคชันโปรแกรมพยายามใส่ Item ที่มีแล้วใน Form นี้หรือ ในฟอร์มอื่นๆ ลงไปในอีก Form หนึ่งก็จะเกิด `IllegalStateException` ขึ้น ดังนั้นจะต้องย้าย Item ออกจาก Form ที่อยู่เดิมก่อนจะสามารถใส่เข้าไปใน Form ใหม่ได้

4.2.4.1 Item เป็นซูปเปอร์คลาสสำหรับคอมโพเนนต์ต่างๆที่สามารถใส่ลงไปใน Form ได้ แต่ละ Item จะมีป้ายข้อความกำกับ ซึ่งเป็นฟิลด์สตริงข้อความที่ติดอยู่กับ Item โดยปกติจะแสดงไว้ใกล้ๆ กับคอมโพเนนต์ที่ปรากฏในหน้าจอ



รูปที่ 4.5 โครงสร้างลำดับชั้นของคลาส Item

4.2.4.1.1 StringItem ประกอบด้วยป้ายและสตริงข้อความที่แสดงผลอย่างเดียวย ผู้ใช้ไม่สามารถแก้ไขค่าใน StringItem ได้โดยตรง ต้องทำผ่านแอปพลิเคชันเท่านั้น โดยใช้เมธอด setText() การสร้างออบเจกต์ของ StringItem ทำได้ดังนี้ StringItem(String label, String text)

4.2.4.1.2 ImageItem อาจมีออบเจกต์ Image ซึ่งสามารถเปลี่ยนแปลงได้ การสร้างออบเจกต์ ImageItem ทำได้ดังนี้ ImageItem(String label, Image img, int layout, String altText)

4.2.4.1.3 ChoiceGroup คือ Item ที่แสดงผลแบบ Choice มีลักษณะคล้ายกับ List แต่ไม่มีแบบ implicit-choice การแสดงผลแบบ exclusive หรือ multiple-choice ของ ChoiceGroup จะเหมือนกับ List การสร้าง ChoiceGroup ทำได้ดังนี้ ChoiceGroup(String label, int choiceType)

4.2.4.1.4 Gauge เป็น Item ที่แสดงผลแบบกราฟแท่งโดยมีค่าระหว่าง 0 ถึง maxValue การสร้างออบเจกต์ Gauge ทำได้ดังนี้ Gauge(String label, boolean interactive, int maxValue, int initialValue)

4.2.4.1.5 DateField คือ Item ที่แก้ไขได้ ใช้สำหรับแสดงค่าวัน เดือน ปีและเวลา สามารถสร้างออบเจกต์ DateField โดยใช้ DateField(String label, int mode, TimeZone timeZone)

4.2.4.1.6 TextField เป็น Item ที่มีสตริงข้อความซึ่งผู้ใช้สามารถแก้ไขได้ การสร้างออบเจกต์ TextField ทำได้ดังนี้ TextField(String label, String text, int maxSize, int constraints)

ข้อแตกต่างระหว่าง TextField และ TextBox คือ TextBox เป็นซับคลาสของ Screen ดังนั้นจึงสามารถแสดงผลบนหน้าจอได้ทันที ทว่า TextField เป็นซับคลาสของ Item จะสามารถแสดงผลได้ก็ต่อเมื่อนำไปไว้ใน Form

4.2.4.2 Form เป็นคอมโพเนนต์ที่สำคัญที่สุดในการพัฒนาส่วนติดต่อกับผู้ใช้สำหรับอุปกรณ์ MIDP เพราะช่วยให้ผู้พัฒนาสามารถสร้างหน้าจอได้อย่างยืดหยุ่น

ในการนำ MIDP มาใช้งาน ได้ระบุการวางโครงร่างของ Item ใน Form สำหรับอุปกรณ์ต่างๆ ไปไว้ว่าต้องเป็นแนวตั้ง Item ที่โฟกัสได้เช่น TextField, DateField, Gauge หรือ ChoiceGroup จะขึ้นบรรทัดใหม่เสมอ สำหรับ StringItem และ ImageItem ซึ่งไม่เกี่ยวข้องกับการโต้ตอบของผู้ใช้จะวางในแนวนอน นอกเสียจากว่าสตริงหรือคำสั่ง Layout ของ ImageItem จะกำหนดให้ขึ้นบรรทัดใหม่ ข้อความใน StringItem หรือ ภาพใน ImageItem จะถูกตัดขึ้นบรรทัดใหม่(กรณีข้อความ) หรือถูกคลิปบางส่วนออก(กรณีภาพ) เพื่อให้พอดีกับความกว้างของหน้าจอ หากจำเป็นต้องเลื่อนจอ ทำได้เฉพาะในแนวตั้งเท่านั้น

4.2.5 อีเวนต์ระดับสูงและการจัดการอีเวนต์ โมเดลการจัดการอีเวนต์ระดับสูงเป็นแบบ delegation-based เหมือนกับใน AWT ของ J2SE ในโมเดลนี้ประกอบด้วย 2 ส่วนคือ แห้งก้านีอีเวนต์ และตัว event listener แห้งก้านีอีเวนต์จะสร้างอีเวนต์แล้วส่งไปยัง event listener จากนั้นเมื่อดีที่จัดการอีเวนต์ของ listener ก็จะมีผลอีเวนต์นั้น

ออบเจ็กต์ของส่วนติดต่อกับผู้ใช้ใน MIDP จะใช้อีเวนต์ระดับสูง 2 ตัวคือ อีเวนต์ command และ ItemStateChanged ดังนั้นจึงมีอีเวนต์ listener 2 ตัวคือ commandListener และ ItemStateListener ตามลำดับ

4.2.5.1 Command เป็นโครงสร้างที่เก็บความหมายของแอ็กชันหรืออีเวนต์ แอ็กชันจริง (หรือการจัดการอีเวนต์) ซึ่งเกิดเมื่อมี command เกิดขึ้นได้กำหนดไว้ในอินเทอร์เฟซของ CommandListener เมื่อ command เริ่มต้นจะเกิดอีเวนต์แล้วส่งไปยังออบเจ็กต์ CommandListener ที่รีจิสเตอร์ไว้ ออบเจ็กต์ Command มีข้อมูล 3 อย่างคือ เลเบล , ชนิด และ ลำดับความสำคัญ

4.2.5.2 ItemStateChanged เมื่อ Form แสดงผลบนจอ ผู้ใช้สามารถเลื่อนโฟกัสจาก Item หนึ่งไปยังอีก Item หนึ่งภายใน Form ได้ โดยไม่เกิดอีเวนต์ให้แอปพลิเคชันสามารถตรวจเห็นได้ แต่หากผู้ใช้แก้ไขเปลี่ยนแปลงสถานะของ Item ที่ได้ตอบโต้ซึ่งอยู่ภายใน Form จะทำให้เกิด ItemStateChanged ขึ้น แต่ถ้าเป็นการเปลี่ยนแปลงค่า Item โดยแอปพลิเคชันด้วยเมธอด setString() ก็จะไม่ทำให้เกิดอีเวนต์ ItemStateChanged

4.3 การใช้ API ระดับล่างในการพัฒนาส่วนติดต่อกับผู้ใช้

API ระดับล่างจะช่วยให้แอปพลิเคชันสามารถควบคุมการแสดงผลบนจอและเข้าถึงอีเวนต์ระดับล่างได้ แต่ก็อาจทำให้แอปพลิเคชันที่พัฒนาด้วย API ระดับล่างที่นำคุณสมบัติเฉพาะอุปกรณ์มาใช้ขาดความยืดหยุ่น และไม่อาจย้ายไปใช้บนแพลตฟอร์มอื่นได้

API ระดับล่างประกอบด้วย 2 ส่วนคือ คลาส Canvas ซึ่งมีชุดคำสั่งทางกราฟิกและเป็นคลาสที่สร้างอีเวนต์ระดับล่างและคลาส Graphics ซึ่งใช้สร้างข้อความและรูปภาพหรือรูปร่างต่างๆ

4.3.1 พื้นฐานเกี่ยวกับ Canvas เป็นซับคลาสของ Displayable ซึ่งใช้ API ระดับล่าง คลาสนี้มีเมธอด paint สำหรับการวาดและเมธอดอื่นๆ สำหรับจัดการอีเวนต์ระดับล่างซึ่งแอปพลิเคชันสามารถเขียนใหม่ได้ เมธอด Paint() ในคลาส Canvas ถูกประกาศเป็น Abstract เช่นเดียวกับคลาส Canvas ดังนั้นแอปพลิเคชันจะต้องการขยายคลาส Canvas เพื่อใช้งานและจะต้องมีเมธอด Paint() เมื่อเป็นซับคลาสของ Canvas แต่เมธอดสำหรับจัดการอีเวนต์ระดับล่างไม่ได้ถูกประกาศเป็น Abstract จึงมีค่าว่าง(นั่นคือ จะไม่ทำงานใดๆ) ดังนั้น แอปพลิเคชันไม่จำเป็นต้องมีเมธอดการจัดการอีเวนต์ครบทั้งหมด หากแต่ต้องการเฉพาะอีเวนต์ที่แอปพลิเคชันสนใจเท่านั้น

ขนาดที่แท้จริงของพื้นที่แสดงผลจะแตกต่างกันไปตามอุปกรณ์แต่ละชนิด แอปพลิเคชันไม่ควรกำหนดขนาดที่ตายตัวแต่ควรเช็คขนาดของพื้นที่แสดงผลโดยใช้เมธอด getHeight() และ getWidth()

จุดประสงค์หลักในการใช้คลาส Canvas ก็เพื่อเข้าถึงและจัดการอีเวนต์ระดับล่าง แต่อีเวนต์ระดับสูงก็สามารถเกิดจากออบเจ็กต์ Canvas ได้ด้วยเช่นกัน แอปพลิเคชันสามารถเพิ่มคอมมานต์ใน

Canvas ได้โดยใช้เมธอด `addCommand()` เหมือนกับซัปคลาสของ `Displayable` อื่นๆ และยังสามารถรีจิสเตอร์ `CommandListener` ใน Canvas ด้วยเมธอด `setCommandListener()` ได้เช่นกัน

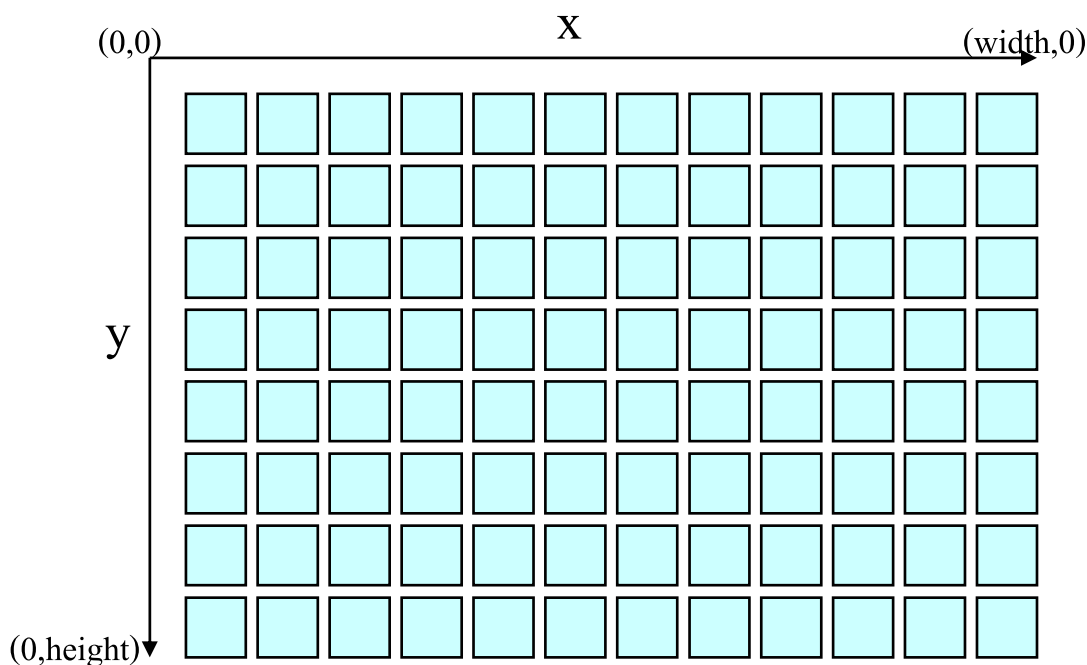
4.3.2 กราฟิก ทำหน้าที่สร้างภาพเรขาคณิต 2 มิติ คลาสนี้คล้ายกับ `java.awt.Graphics` ใน J2SE โหมดในการวาดของคลาส `Graphics` เท่านั้นที่เป็นแบบแทนที่พิกเซล พิกเซลที่มีอยู่เดิมบนจอจะถูกแทนที่ด้วยพิกเซลปัจจุบันที่ระบุในออบเจกต์ `Graphics` ในคลาส `Graphics` ไม่มีฟังก์ชันอื่นที่ทำหน้าที่รวมค่าของพิกเซล อาทิ คำสั่งเกี่ยวกับบิตแมพหรือการผสม/ ซ้อนภาพแบบแอลฟา (การทำให้ฉากหลังโปร่งใสเพื่อซ้อนทับอีกภาพ)

ออบเจกต์ `Graphics` สามารถวาดบนจอโดยตรงหรือจะวาดลงในบัฟเฟอร์ของรูปที่ไม่อยู่บนจอก็ได้ ฉากที่ออบเจกต์ `Graphics` วาดลงไปนั้นจะเป็นอะไรก็ขึ้นอยู่กับแหล่งกำเนิดของออบเจกต์ `Graphics`

วิธีเดียวที่จะดึงออบเจกต์ `Graphics` มาวาดได้ ทำได้โดยใช้เมธอด `paint(Graphics g)` ของออบเจกต์ `Canvas` แอปพลิเคชันสามารถวาดเนื้อหาบนออบเจกต์ `Canvas` ด้วยการใช้ออบเจกต์ `Graphics` นี้เท่านั้น

แอปพลิเคชันสามารถดึงออบเจกต์ `Graphics` เพื่อวาดลงในบัฟเฟอร์ของรูปได้โดยใช้เมธอด `getGraphics()` แอปพลิเคชันสามารถเก็บออบเจกต์ `Graphics` ที่ดึงมาได้ตลอดและเรียกใช้คำสั่งในการวาดได้ตลอดเวลา

ระบบพิกัดของกราฟิกมีจุดปัก (เรียกว่าจุด anchor) เริ่มที่มุมบนซ้ายของพื้นที่วาด ค่า x จะเพิ่มจากซ้ายไปขวา และค่า y จะเพิ่มจากบนลงล่าง

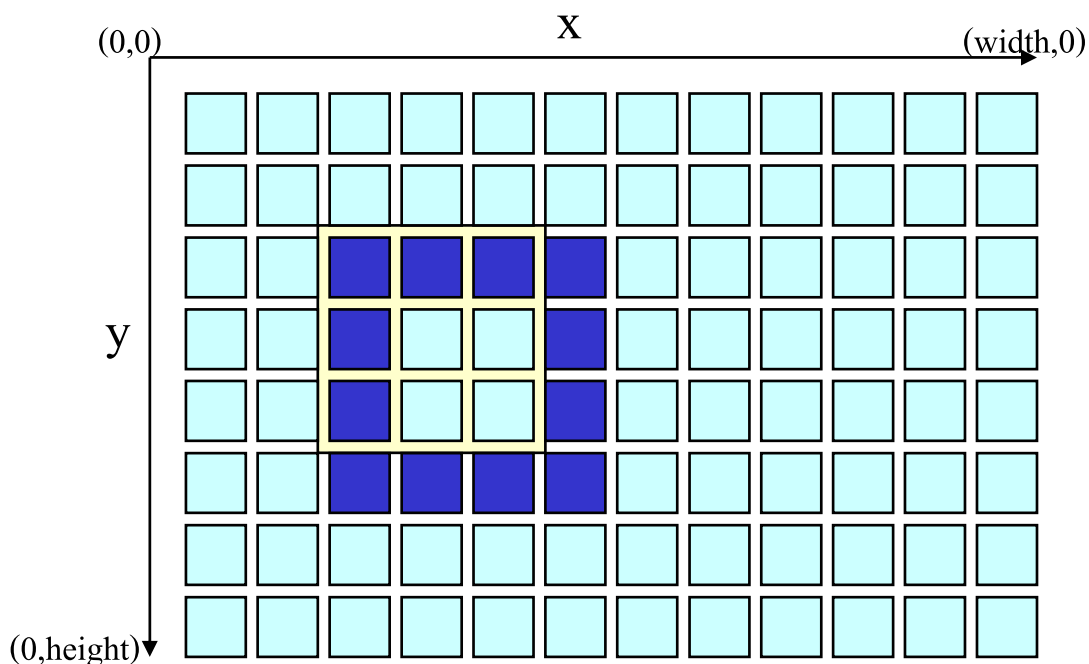


รูปที่ 4.6 ระบบพิกัดของ *Graphic*

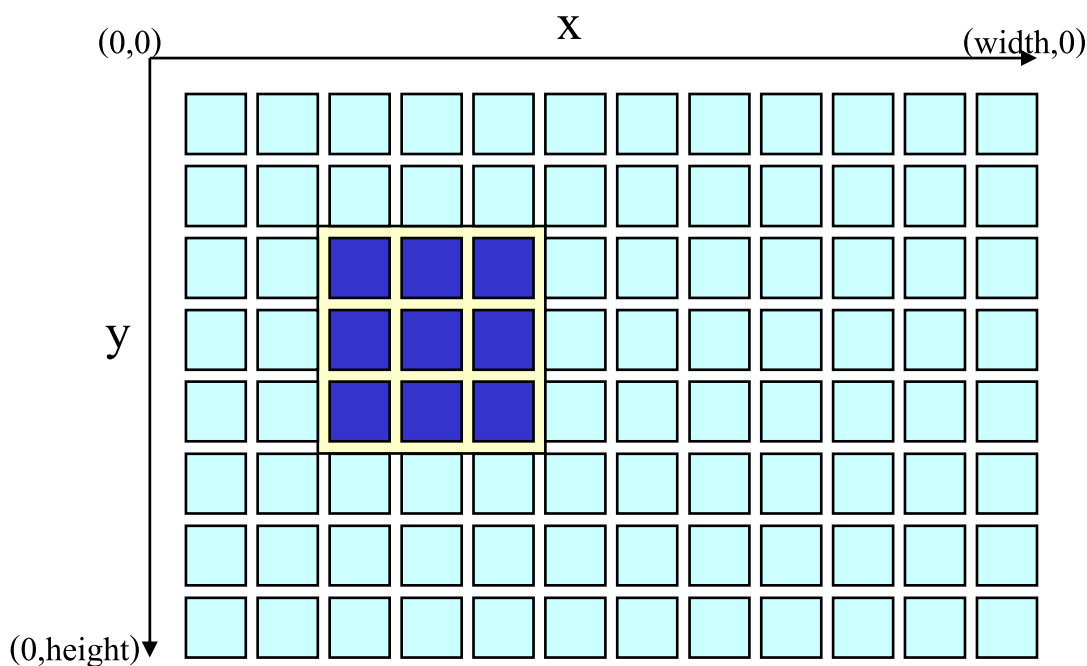
ระบบพิกัดจะอ้างถึงตำแหน่งระหว่างพิกเซล ไม่ใช่ตำแหน่งของพิกเซลเอง ตัวอย่างเช่น พิกเซลแรกที่มีมุมบนซ้ายสุดจะอยู่ในช่องสี่เหลี่ยมที่ล้อมรอบด้วยพิกัด (0,0),(1,0),(0,1),(1,1)

คำสั่งวาดหลายเส้น โครงร่างจะเดินไปตามพิกัดโดยเสมือนมีปากกาติดอยู่ข้างล่างและข้างขวาของเส้นทางที่เดินขนาดของปากกาคือกว้าง 1 พิกเซล ยาว 1 พิกเซล ดังนั้นพิกเซลที่อยู่ทางขวาหรือใต้พิกัดของเส้นทางวาดจะเป็นเป้าหมายของคำสั่ง ตัวอย่างเช่นการเรียก drawRect(2,2,3,3) จะได้ผลดังรูป 4.7

คำสั่งการเติมภาพจะแตกต่างจากคำสั่งการวาด เฉพาะพิกเซลที่ถูกปิดล้อมด้วยพิกัดของเส้นทางวาดเท่านั้นที่จะได้รับผลคำสั่ง ตัวอย่างเช่น fillRect(2,2,3,3) จะได้ผลดังรูป 4.8



รูปที่ 4.7 เส้นทางการวาด



รูปที่ 4.8 การเติมภาพ

คลาสกราฟิกจะใช้พารามิเตอร์สำหรับวาดรูปเหมือนกัน พารามิเตอร์เหล่านั้นได้แก่ สี ฟอนต์ และลายเส้น

API ระดับล่างใช้ระบบสีแบบ 24 บิต นั่นคือใช้ 8 บิตสำหรับแต่ละองค์ประกอบของสี คือ สีแดง สีเขียว และสีน้ำเงิน เนื่องจากอุปกรณ์บางชนิดอาจไม่สนับสนุนสีแบบ 24 บิต การนำไปใช้งานจึงต้องมีการจับคู่สีที่แอปพลิเคชันต้องการใหม่ให้เป็นสีที่อุปกรณ์สามารถแสดงผลได้ แอปพลิเคชันสามารถตรวจสอบว่าอุปกรณ์สามารถแสดงผลสีหนึ่งๆ ได้หรือไม่ด้วยเมธอด `Display.isColor()` และดูจำนวนสี (ถ้า `isColor()` เป็น true) หรือระดับเฉดเทา (ถ้า `isColor()` เป็น false) ด้วยเมธอด `Display.numColors()`

โดยปกติแล้วก่อนที่จะแอปพลิเคชันจะวาดเนื้อหาบนจอหรือบนรูปภาพ จะต้องกำหนดสีของ foreground และ background ก่อนใน J2SE มีเมธอด `setBackground()` และ `setForeground()` เพื่อใช้สำหรับงานนี้ แต่ในคลาส `Graphics` มีเพียงเมธอด `setColor()` เท่านั้น การเซ็ตสี background ต้องใช้วิธี `setColor()` เพื่อให้ได้สีที่ต้องการ จากนั้นจึงใช้วิธีเติมสีรูปสี่เหลี่ยมด้วย `fillRect()` แล้วจึงค่อยเซ็ตสีที่ต้องการสำหรับ foreground

คลาส `Graphics` มีขั้นตอนสำหรับการวาดรูปและเติมรูปสี่เหลี่ยมอยู่ 2 ชุด ชุดแรกเป็นการวาดสี่เหลี่ยมที่มีมุมเหลี่ยม

```
void drawRect(int x, int y, int width, int height)
```

```
void fillRect(int x, int y, int width, int height)
```

ชุดที่ 2 เป็นการวาดสี่เหลี่ยมที่มีมุมโค้งมน

```
void drawRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight)
```

```
void fillRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight)
```

คลาส Graphics มีเมธอดดังต่อไปนี้สำหรับวาดและเติมเส้นโค้งหรือรูปส่วนโค้งซึ่งระบุขอบเขตด้วยพิกัดของรูปสี่เหลี่ยม

```
void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)
```

```
void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle)
```

คลาส Graphics มีเมธอดสำหรับวาดข้อความดังต่อไปนี้

```
void drawChar(char character, int x, int y, int anchor)
```

```
void drawChars(char[] data, int offset, int length, int x, int y, int anchor)
```

```
void drawstring(String str, int x, int y, int anchor)
```

```
void drawSubstring(String str, int offset, int len, int x, int y, int anchor)
```

นอกจากสีและลายเส้นแล้ว ฟอนต์ก็เป็นพารามิเตอร์อีกตัวหนึ่งที่ออบเจกต์ Graphics สามารถกำหนดได้

คลาส Font เป็นตัวแทนของฟอนต์และเมตริกซ์ของฟอนต์ แอปพลิเคชันไม่สามารถสร้างออบเจกต์ของฟอนต์ใหม่ได้ ต้องใช้ฟอนต์ที่มีอยู่เดิม ซึ่งสามารถเรียกดูรายละเอียดต่างๆ ของฟอนต์ได้โดยใช้เมธอดดังนี้

```
Static Font getFont(int face, int style, int size)
```

การวาดข้อความจะขึ้นอยู่กับจุดปักหรือ anchor ใช้ลดปริมาณการคำนวณค่าเมื่อแอปพลิเคชันต้องการวางข้อความ

จุดปักมีค่าคงที่ 3 ตัวในแนวนอนได้แก่ LEFT, HCENTER และ RIGHT และ 4 ตัวในแนวตั้งได้แก่ TOP, BASELINE, VCENTER และ BOTTOM การกำหนดจุดปักต้องระบุด้วยค่าคงที่ในแนวนอน 1 ค่ารวมด้วยโอเปอเรเตอร์ OR กับค่าคงที่ในแนวตั้ง 1 ค่า อาทิ TOP, BASELINE และ BOTTOM แต่ไม่สามารถใช้ค่า VCENTER กับพารามิเตอร์ของการวาดข้อความที่จุดปัก ทั้งนี้ เนื่องจากการวางตำแหน่งตรงกลางในแนวตั้งไม่มีประโยชน์กำหนดยาก และนำไปใช้งานจริงได้ลำบาก

คลาส Graphics มีเมธอดเดียวสำหรับการวาดรูป

```
void drawImage(Image img, int x, int y, int anchor)
```

รูปที่วาดอาจจะแก้ไขได้หรือไม่ได้ ตำแหน่งจุดปักจะใช้ค่าคงที่เหมือนกับที่อธิบายในเรื่องการวาดข้อความ ข้อแตกต่างประการเดียว คือ จุดปัก BASELINE ของข้อความจะถูกแทนที่ด้วย VCENTER ของการวางรูปภาพ

4.3.3 อีเอนต์ระดับล่างและการจัดการกับอีเอนต์ คลาส Canvas ใช้ API ส่วนติดต่อกับผู้ใช้ระดับล่าง และรับผิดชอบการจัดการอีเอนต์ในระดับล่าง เช่น อีเอนต์ปุ่ม(คีย์) และอีเอนต์ตัวชี้หรือพอยน์เตอร์ (สำหรับอุปกรณ์บางรุ่น)

ใช้โมเดลการจัดการอีเอนต์แบบเดียวกับ AWT ของ J2SE นั่นคือ ทุกอีเอนต์จะถูกส่งไปที่ Canvas และแอปพลิเคชันจะทำหน้าที่คัดเอาอีเอนต์ที่ไม่ต้องการออกไป

4.3.3.1 อีเอนต์ Show และ Hide เมธอด `showNotify()` ถูกเรียกขึ้นมาก่อนออบเจกต์ Canvas จะแสดงผลบนจอ และเมธอด `hideNotify()` ถูกเรียกหลังจากที่ออบเจกต์ Canvas ถูกย้ายออกไปจากจอแล้ว การเปลี่ยนสถานการณ์มองเห็น/ไม่เห็นของ Canvas อาจเกิดจากซอฟต์แวร์แอปพลิเคชันที่ย้าย MIDlet สลับไปมาระหว่าง foreground กับ background หรืออาจเกิดจากข้อความของระบบโพล์ขึ้นมายัง Canvas ดังนั้น `showNotify()` และ `hideNotify()` จึงไม่อยู่ภายใต้การควบคุมของ MIDlet และอาจเกิดอีเอนต์เหล่านี้ขึ้นได้บ่อยๆ

เมธอดของอีเอนต์ Key และ Pointer เมธอด `paint()` `commandAction()` เหล่านี้ จะถูกเรียกก็ต่อเมื่อสามารถมองเห็นออบเจกต์ Canvas ได้บนอุปกรณ์ นั่นคือจะเรียกเมธอดเหล่านี้ได้หลังจาก `showNotify()` ถูกเรียกขึ้นมาแล้วและก่อนจะเรียก `hideNotify()` เท่านั้น เมื่อ `hideNotify()` ถูกเรียก เมธอดของอีเอนต์ Key และ Pointer เมธอด `paint()` `commandAction()` จะไม่ถูกเรียกจนกว่า `showNotify()` ถัดไปจะถูกเรียก

4.3.3.2 อีเอนต์ Key แอปพลิเคชันจะได้รับอีเอนต์ Key เมื่อผู้ใช้กดปุ่มใดๆบนแป้น

4.3.3.2.1 Key Code และ Key Name ปุ่มแต่ละปุ่มบนจอจะมีค่ารหัสหรือ key code กำหนดไว้ซึ่งเป็นค่าที่ไม่ซ้ำกัน เว้นแต่ปุ่มที่มีค่า key code ซ้ำกันนั้นสามารถใช้แทนกันได้ MIDP กำหนดค่า key code ต่อไปนี้สำหรับปุ่มบนแป้นโทรศัพท์มาตรฐาน KEY_NUM0, KEY_NUM1, KEY_NUM2, KEY_NUM3, KEY_NUM4, KEY_NUM5, KEY_NUM6, KEY_NUM7, KEY_NUM8, KEY_NUM9, KEY_STAR และ KEY_POUND

ค่ารหัสเหล่านี้มีค่าเทียบได้กับรหัสยูนิโค้ดของตัวอักษร เช่น KEY_NUM0 หรือปุ่มเลข 0 บนแป้นกด จะมีค่ารหัสเป็น 48 นอกจากปุ่มเหล่านี้แล้วบนแป้นกดอาจมีปุ่มอื่นๆ key code นอกเหนือจากที่ระบุไว้ข้างต้นจะมีค่าติดลบเพื่อให้แตกต่างจากปุ่มมาตรฐานที่เป็นค่าบวก

แต่ละปุ่มจะมีชื่อหรือ key name ชื่อของปุ่มสามารถดูได้จาก `getKeyName()` ของคลาส Canvas

เมธอดในการจัดการอีเอนต์ของ key มี 3 เมธอดด้วยกัน

`protected void keyPressed(int keyCode)`

`protected void keyReleased(int keyCode)`

`protected void keyRepeated(int keyCode)`

อีเอนต์ key-repeated จะเกิดขึ้นเมื่อผู้ใช้กดปุ่มติดๆกันสองครั้งในช่วงเวลาสั้นๆ อุปกรณ์บางชนิดสามารถจัดการอีเอนต์ key-repeated ได้ บางชนิดจัดการไม่ได้ วิธีตรวจสอบว่าอุปกรณ์รู้จักอีเอนต์ key-repeated หรือไม่ทำได้โดย

```
public boolean hasRepeatEvents()
```

การเปลี่ยนแปลงที่เกิดกับชั้นคลาสของ Screen จะแสดงบนจอโดยอัตโนมัติ แอปพลิเคชันไม่จำเป็นต้องปรับปรุงหน้าจอ แต่สำหรับคลาส Canvas หน้าจอไม่สามารถปรับปรุงโดยอัตโนมัติได้เอง แอปพลิเคชันจะต้องใช้เมธอด repaint() เพื่อปรับปรุงหน้าจอเมื่อมีการเปลี่ยนแปลง

4.3.3.2 แอ็กชันเกม แอปพลิเคชันหลายตัวร้องการอีเวนต์ของปุ่มลูกศรและอีเวนต์อื่นๆ สำหรับเล่นเกม MIDP กำหนดค่าของปุ่มเหล่านั้นดังนี้ UP, DOWN, LEFT, RIGHT, FIRE, GAME_A, GAME_B, GAME_C และ GAME_D แต่ละปุ่มจะจับคู่กับแอ็กชันของเกม 1 แอ็กชัน อย่างไรก็ตามแอ็กชันเกมหนึ่งๆ อาจใช้ปุ่มรวมกันมากกว่า 1 ปุ่ม ค่า key code และแอ็กชันเกมสามารถแปลงกลับไปมาได้โดยใช้เมธอดต่อไปนี้

```
int getKeyCode(int gameAction)
```

```
int getGameAction(int keyCode)
```

วิธีการจับคู่แอ็กชันเกมกับ key code จะขึ้นอยู่กับระบบ เช่น ในอุปกรณ์บางตัว แอ็กชัน UP, DOWN, LEFT, RIGHT อาจจับคู่กับปุ่มลูกศรทั้งสี่ แต่อุปกรณ์อีกตัวอาจจับคู่ไปเป็นปุ่มตัวเลข 2, 4, 6, 8 เป็นต้น แอปพลิเคชันควรใช้วิธีแปลงค่า key code เป็นแอ็กชันเกมด้วย getGameAction() เพื่อให้สามารถโยกย้ายไปใช้กับอุปกรณ์อื่นๆ ได้ง่าย

4.3.3.3 อีเวนต์ Pointer สำหรับอุปกรณ์ที่มีจอแบบสัมผัส พอยน์เตอร์ (Pointer) หรือตัวชี้อาจเป็นวัตถุใดๆ ที่ใช้ควบคุมจอภาพ เช่น อุปกรณ์ชี้ ปากกาหรือแม้แต่นิ้วมือ อีเวนต์ของ pointer มี 3 แบบได้แก่ pointer pressevent, pointer release event และ pointer drag event

เมื่อมีออบเจกต์ Screen อยู่บนหน้าจอ การเปลี่ยนแปลงใดๆ ที่เกิดกับออบเจกต์ Screen จะส่งผลกระทบต่อหน้าจอโดยอัตโนมัติ แอปพลิเคชันไม่จำเป็นต้องปรับปรุงหน้าจอเอง แต่ถ้าเป็นออบเจกต์ Canvas อยู่บนจอ แอปพลิเคชันต้องจัดการปรับปรุงหน้าจอ เมื่อมีการเปลี่ยนแปลงบน Canvas โดยมีเมธอดในการปรับปรุงหน้าจอ 2 เมธอดดังนี้

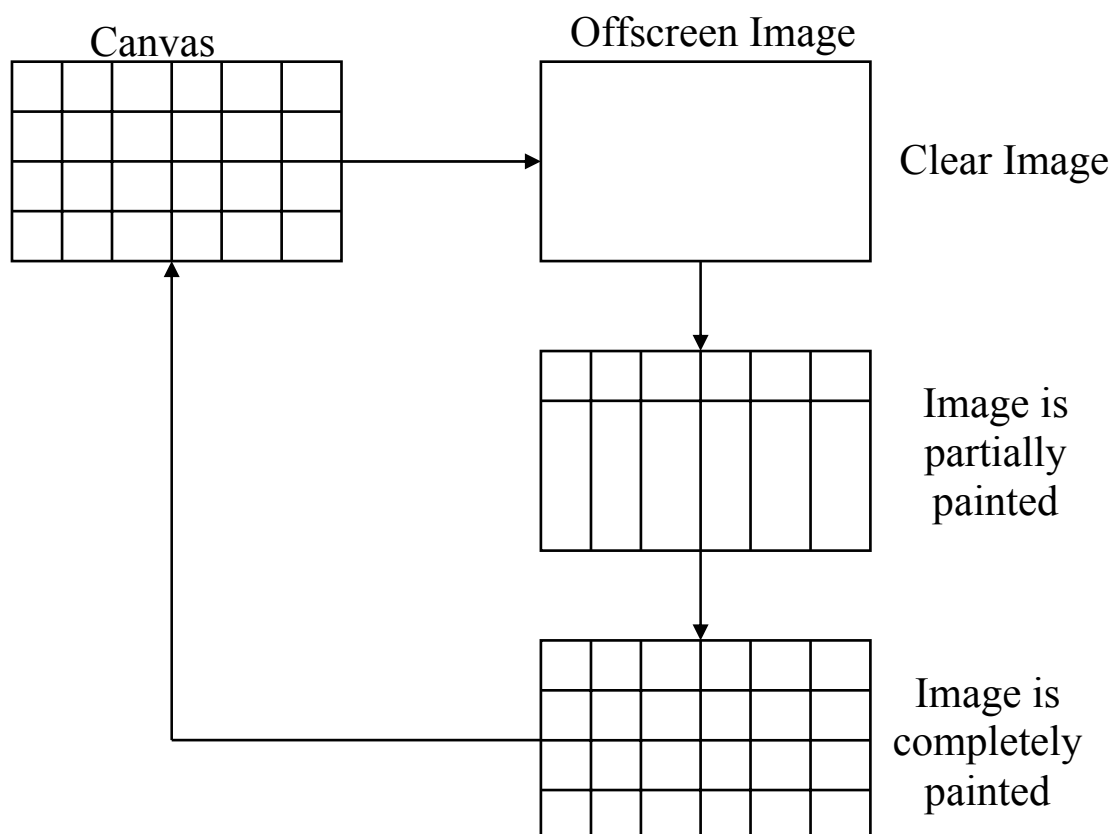
```
public final void repaint(int x, int y, int width, int height)
```

```
public final void repaint()
```

ถ้า Canvas อยู่ในสถานะที่มองไม่เห็น การเรียกเมธอดทั้งสองนี้ก็จะไม่เกิดผลใดๆ

เมื่อความเร็วในการทำงานของ KVM(K Virtual Machine) ช้าลง บางครั้งอาจสังเกตเป็นภาพกระตุกซึ่งมีสาเหตุมาจากการที่ MIDlet ต้องล้างพื้นที่วาดด้วยสี background ก่อนที่เนื้อหาใหม่จะถูกวาดลงไป วิธีแก้ทำได้โดยใช้เทคนิคบัพเฟอร์ 2 ชั้น

เทคนิคบัพเฟอร์ 2 ชั้น มีหลักการง่ายๆ คือ แทนที่จะวาดหรือล้างเนื้อหาบนพื้นที่ Canvas โดยตรง เราจะสร้างภาพที่อยู่นอก Screen ขึ้นมาแทนโดยเป็นเสมือนอีกสำเนาหนึ่งของ Canvas การวาดหรืออัปเดต Canvas จะเอาบัพเฟอร์นี้เป็นพื้นที่ใช้งาน เมื่อเสร็จแล้วก็คัดลอกไปทับพื้นที่ Canvas จริงโดยไม่ต้องรอให้มีการเคลียร์ Screen ก่อน



รูปที่ 4.9 ขั้นตอนการอัปเดตหน้าจอด้วย offscreen image

MIDP บางตัวใช้เทคนิคแบบนี้สำหรับออบเจกต์ Graphic ที่อยู่บน Canvas คุณสามารถตรวจสอบว่าออบเจกต์ Graphics ใน Canvas ใช้เทคนิคนี้อยู่หรือไม่ด้วย

```
public boolean isDoubleBuffered()
```

สามารถเขียนโปรแกรมโดยใช้เทคนิคบัฟเฟอร์ 2 ชั้นในแอปพลิเคชันได้ ชั้นแรกสร้างรูปที่อยู่นอก Screen ที่มีขนาดเท่า Canvas โดยใช้

```
buffer_image = Image.createImage(width, height)
```

จากนั้นก็ดึงออบเจกต์ Graphics เพื่อมาวาด

```
bg = buffer_image.getGraphics()
```

สามารถดึงเอาออบเจกต์ของ Graphics ที่อยู่นอก Screen ได้โดยใช้ getGraphics() ของรูปนั้น และจะยังคงอยู่ตลอดเวลาที่แอปพลิเคชันทำงาน ถ้าออบเจกต์ Graphics ไม่ถูกทำลายหลังจากการใช้งานก็จะทำให้สิ้นเปลืองหน่วยความจำ ดังนั้นหลังจากใช้ออบเจกต์ Graphics แล้วต้องทำลายด้วย

```
bg = null
```

ด้วยการใช้เทคนิคบัฟเฟอร์ 2 ชั้น ทำให้แอปพลิเคชันสามารถขจัดปัญหาภาพกระตุกได้ อย่างไรก็ตามที่ตามมาคือแอปพลิเคชันจะตอบสนองต่ออีเวนต์ Key ช้าลงเพราะต้องคัดลอกรูปที่อยู่นอกจอมายัง Canvas

นอกจากนี้ เทคนิคฟลैตฟอร์ 2 ชั้น จะใช้หน่วยความจำมากและอุปกรณ์บางชนิดอาจไม่ได้เพื่อไว้วิธีแก้ไขปัญหภาพระตุกอีกทางเลือกหนึ่งคือค่อยๆ ลบส่วนของหน้าจอที่เปลี่ยนแปลงอย่างระมัดระวังแทนที่จะลบทั้งหน้าจอ

4.4 หน่วยเก็บข้อมูลแบบคงตัว

J2ME MIDP มีระบบฐานข้อมูลแบบเรกคอร์ดง่ายๆ ที่เรียกว่า record management system (RMS) ตารางในฐานข้อมูลจะเรียกว่าเป็น record store และแต่ละรายการข้อมูลใน record store จะเรียกว่า 'เรกคอร์ด' ชุดคำสั่ง API สำหรับสร้าง เข้าถึง และจัดการกับเรกคอร์ดต่างๆ นั้นจะอยู่ในแพ็คเกจ javax.microedition.rms

4.4.1 Record Stores rms ที่มีใน MIDP เป็นระบบฐานข้อมูลแบบง่ายๆ record store ซึ่งเทียบเท่ากับตารางในฐานข้อมูล คือไฟล์ที่ประกอบด้วยชุดเรกคอร์ด รายละเอียดของ record store จะกำหนดในคลาส RecordStore class

4.4.2 เรกคอร์ด กลุ่มของข้อมูลใน record store เรียกว่าเรกคอร์ด โครงสร้างของเรกคอร์ดเกิดจากอาร์เรย์ของไบต์และจะกำหนดค่าสำหรับแต่ละเรกคอร์ดด้วย RecordId เรกคอร์ดจะสามารถสร้างขึ้นมาใหม่ ลบ เรียกดู หรือแก้ไขได้ก็ต่อเมื่อได้เปิด record store ด้วยเมธอดใน API ดังนี้

```
public int addRecord(byte[] data, int offset, int numBytes)
```

```
public void deleteRecord(int recordId)
```

```
public int getRecord(int recordId, byte[] buffer, int offset)
```

```
public byte[] getRecord(int recordId)
```

```
public void setRecord(int recordId, byte[] newData, int offset, int numBytes)
```

การเพิ่มเรกคอร์ด เรกคอร์ดแรกที่เราสร้างขึ้นใน record store จะมีค่า RecordID = 1 เรกคอร์ดที่สร้างเพิ่มเติมในภายหลังจะมีค่า recordId เพิ่มขึ้นครั้งละหนึ่งค่า

การลบเรกคอร์ด ทำได้โดย deleteRecord(int recordID) หากเรียกเมธอดนี้โดยไม่เปิด record store จะเกิด RecordStoreNotOpenException หรือหากหาเรกคอร์ดที่ต้องการลบไม่พบ ก็จะเกิด InvalidRecordIDException นอกจากนี้หากมีเอกเซชั่น record store แบบอื่นๆ เกิดขึ้น ก็จะเกิด RecordStoreException ตามมา

ค่า RecordID ของเรกคอร์ดที่ลบออกไปแล้ว จะไม่นำกลับมาใช้อีก เพราะถือว่าเป็นค่าหลักสำหรับเรกคอร์ดใหม่ที่อยู่ถัดไป โดยเรกคอร์ดถัดไปจะยึดค่าเดิมเป็นหลัก และเพิ่มค่าเข้าไปอีกหนึ่งค่าเสมอ

การลบ record ไม่ทำให้ขนาดของ record store ลดลง เพราะมาทำเครื่องหมายพื้นที่ว่างหน้าบล็อกข้อมูลของเรกคอร์ดที่ลบทิ้งไป พร้อมนำไปใช้งานอื่นๆต่อ

4.4.3 RecordEnumeration หลังจากที่เราเรกคอร์ดถูกลบออกไปแล้ว RecordId ใน record store จะไม่เป็นเลขที่ต่อเนื่องกันอีก และแม้จะสามารถอ่านค่าเรกคอร์ดทั้งหมดโดยใช้ MIDlet

การอ่านข้อมูลทั้งหมดใน record store ด้วยวิธีนี้เป็นการอ่านค่าแบบลองผิดลองถูก หากเรคคอร์ดที่มีค่าเท่ากับอยู่จริง ก็จะอ่านค่าเรคคอร์ดโดยใช้ `getRecord()` แต่หากไม่พบเรคคอร์ดนั้นๆ ก็จะได้เกิด `InvalidRecordIDException` การใช้วิธีนี้อ่านค่าจะได้ผลน้อยมาก หากมีการลบเรคคอร์ดใน record store ออกไปแล้วเป็นจำนวนมาก

API ของ RMS ได้เตรียม `RecordEnumeration` ซึ่งเข้าถึงข้อมูลใน record store ได้ดีกว่าไว้ เนื่องจากเป็นคลาสที่เข้าถึงเรคคอร์ดใน record store ได้ถึงสองทิศทางด้วยกัน ทั้งนี้ MIDP จะกำหนด `RecordEnumeration` เป็นอินเตอร์เฟส และปล่อยให้บริษัทผู้ผลิตกำหนดรายละเอียด ดังนั้น ผู้ที่สนับสนุน J2ME MIDP จะต้องนำ `RecordEnumeration` มาใช้งานในอุปกรณ์ของคนเสมอ นักพัฒนาถือว่า `RecordEnumeration` เป็นคลาสหลักคลาสหนึ่ง

โครงสร้างของ `RecordEnumeration` เป็นลักษณะของรายการที่มีปลายเชื่อมต่อ 2 ด้าน โดยแต่ละจุดเชื่อมต่อหนึ่ง `recordId` มีหน้าที่ในการจัดลำดับ `recordId` ของเรคคอร์ดใน record store สามารถสร้าง `RecordEnumeration` ในคลาส `RecordStore` โดย

```
public RecordEnumeration enumerateRecords(RecordFilter filter, RecordComparator comparator, boolean keepUpdated)
```

คลาส `RecordEnumeration` มี API สำหรับการเข้าถึงข้อมูลในเรคคอร์ดของ record store ดังนี้

```
void destroy()
```

```
boolean hasNextElement()
```

```
boolean hasPreviousElement()
```

```
byte[] nextRecord()
```

```
int nextRecord()
```

```
int numRecord()
```

```
byte[] previousRecord()
```

```
int previousRecord()
```

```
void rebuild()
```

```
void reset()
```

เมื่อต้องใช้ตัวแจกแจงหมายเลขหลายครั้ง ต้องใช้ `reset()` เพื่อย้ายจากจุดเชื่อมต่อปัจจุบัน (พอยน์เตอร์) กลับไปตั้งต้นที่ `recordId` แรกในรายการข้อมูลก่อน

4.5 หลักการเขียนโปรแกรมเครือข่ายใน MIDP ของ J2ME

การเขียนโปรแกรมเครือข่าย นำเอาจุดเด่นเรื่องความสามารถในการเชื่อมต่อของอุปกรณ์เหล่านี้มาใช้ มีบทบาทสำคัญในการพัฒนาแอปพลิเคชันสำหรับอุปกรณ์ไร้สาย

4.5.1 การเขียนโปรแกรมเครือข่ายด้วย J2SE และ J2ME

การเขียนโปรแกรมสำหรับเครือข่ายด้วย J2SE จะง่ายและตรงไปตรงมา ซึ่งใน J2SE ได้ให้ไลบรารีของเครือข่ายซึ่งมีฟังก์ชันให้ใช้มากมาย ในแพ็คเกจ `java.io` มีคลาสที่สนับสนุนการนำเข้า -

ส่งออกข้อมูลประมาณ 60 คลาส ขณะที่อีกประมาณ 20 คลาสในแพ็คเกจ java.net สนับสนุนการทำงานด้วยเครือข่าย

อย่างไรก็ดี คลาสส่วนใหญ่ในแพ็คเกจทั้งสองออกแบบมาสำหรับระบบคอมพิวเตอร์ทั่วไปที่มีหน่วยประมวลผล หน่วยความจำ และพื้นที่เก็บข้อมูลที่เพียงพอ ไฟล์คลาสเหล่านี้มีขนาดคงที่ทั้งสิ้นประมาณ 200 กิโลไบต์ ซึ่งใหญ่เกินกว่าที่จะใส่ลงไปในอุปกรณ์ไร้สาย ซึ่งมีทรัพยากรต่างๆ จำกัด และมีพื้นที่เก็บข้อมูลเพียงไม่กี่ร้อยกิโลไบต์

ไม่เฉพาะขนาดของไฟล์เท่านั้นที่เป็นอุปสรรคในการใช้งานบนอุปกรณ์ไร้สาย Java 2 Micro Edition จะต้องรองรับอุปกรณ์เคลื่อนที่ได้หลากหลายประเภท การติดต่อเครือข่ายและไลบรารีของไฟล์ I/O ในแต่ละอุปกรณ์แตกต่างกันมาก เนื่องจากความต้องการขั้นต่ำในการเชื่อมต่อและความสามารถของ I/O ที่แตกต่างกัน ความแตกต่างระหว่างเครือข่ายทำให้ต้องกำหนดสาระสำคัญของการสื่อสาร (abstraction) ในจาวาไลบรารี 2 รูปแบบคือ การเชื่อมต่อแบบดาต้าแกรมสำหรับเครือข่ายสลับกลุ่มข้อมูล และคือ การเชื่อมต่อแบบซ็อกเก็ตสำหรับเครือข่ายแบบสลับวงจร ทั้งนี้ ผู้ผลิตซึ่งสนับสนุนดาต้าแกรมอาจไม่สนใจจะสนับสนุนซ็อกเก็ตก็ได้ และเช่นเดียวกันในกรณีของผู้ผลิตซึ่งสนับสนุนซ็อกเก็ต

การเข้าถึงไฟล์ I/O ในอุปกรณ์ไร้สายก็เป็นเรื่องเฉพาะอุปกรณ์และมีวิธีต่างๆ ในการนำไปใช้เช่นกัน ผู้ผลิตที่สนับสนุนกลไก I/O ประเภทหนึ่งแล้วมักจะไม่นับสนับสนุนกลไกประเภทอื่นๆ อีกเนื่องจากข้อจำกัดด้านหน่วยความจำ

เครือข่ายใน J2ME จะต้องมีความยืดหยุ่นสูงเพื่อรองรับอุปกรณ์หลากหลายประเภท ขณะเดียวกันก็จะต้องสนับสนุนเฉพาะอุปกรณ์ด้วย ดังนั้น จึงได้ริเริ่มนำเอากรอบการติดต่อสื่อสารทั่วไปเข้ามาใช้ใน CLDC แนวคิดของกรอบการสื่อสารทั่วไปคือ การกำหนดสาระสำคัญ (abstraction) ของเครือข่ายและไฟล์ I/O เพื่อสนับสนุนอุปกรณ์พกพาประเภทต่างๆ ให้มากที่สุดเท่าที่จะทำได้ และปล่อยให้การนำสาระสำคัญของการสื่อสารมาใช้งานจริงเป็นหน้าที่ของผู้ผลิตอุปกรณ์แต่ละราย ทั้งนี้ได้กำหนดสาระสำคัญของการสื่อสารจะไว้ในอินเตอร์เฟซของจาวา ผู้ผลิตจะเป็นผู้เลือกว่าจะนำ MIDP หรือ PDAP มาใช้งานโดยพิจารณาตามความสามารถของอุปกรณ์ของตน

4.5.2 กรอบการติดต่อสื่อสารทั่วไป (The Generic Connection Framework)

นำกรอบการติดต่อสื่อสารทั่วไปเข้ามาใช้ใน CLDC ของ J2ME ก็เพื่อที่จะสะท้อนให้เห็นถึงความต้องการเครือข่ายขนาดเล็กและไฟล์ I/O สำหรับอุปกรณ์เคลื่อนที่ประเภทต่างๆ

กรอบการติดต่อสื่อสารทั่วไปสร้างฟังก์ชันเครือข่ายและคลาสไฟล์ I/O จากแพ็คเกจ java.io และ java.net ของ J2SE หรือเป็นซับเซตของคลาสใน J2SE แต่มีขนาดเล็กกว่า คลาสและอินเตอร์เฟซทั้งหมดของ J2ME รวมอยู่ในแพ็คเกจเดียว คือ javax.microedition.io

เพื่อให้กรอบการติดต่อสื่อสารทั่วไปมีความยืดหยุ่นและสนับสนุนอุปกรณ์ได้หลากหลายตามที่ต้องการ จะนำสาระสำคัญในการติดต่อสื่อสารรูปแบบต่างๆมาใช้ ได้แก่ อินเตอร์เฟซการติดต่อ 7 ลักษณะ คือ Connection, ContentConnection, DatagramConnection, InputConnection, OutputConnection, StreamConnection และ StreamConnectionNotifier

4.5.3 การเขียนโปรแกรมเครือข่ายบนอุปกรณ์ไร้สายโดยการเชื่อมต่อแบบซ็อกเก็ต

ซ็อกเก็ตเป็นกลไกการติดต่อสื่อสารระหว่างโปรแกรมที่ทำงานบนเครือข่ายปลายทาง 2 แห่ง อาทิ อุปกรณ์ไร้สายกับเซิร์ฟเวอร์ระยะไกล หรือระหว่างอุปกรณ์ไร้สาย 2 อุปกรณ์ มีประสิทธิภาพน่าเชื่อถือ ความสามารถในการเชื่อมต่อแบบซ็อกเก็ตในอุปกรณ์เคลื่อนที่บางประเภททำให้เกิดแอปพลิเคชันที่เขียนด้วย J2ME บนไคลเอนต์และเซิร์ฟเวอร์ต่างๆ การเชื่อมต่อแบบซ็อกเก็ตกับเซิร์ฟเวอร์ SMTP และ POP3 ทำให้ไคลเอนต์อีเมลบนอุปกรณ์ไร้สายสามารถรับและส่งอีเมลข้อความผ่านระบบอินเทอร์เน็ตได้ตามปกติ

ซ็อกเก็ตช่วยนักพัฒนา J2ME สามารถพัฒนาแอปพลิเคชันทุกชนิดเพื่อใช้งานบนอุปกรณ์ไร้สายได้ อย่างไรก็ตามผู้ผลิตอุปกรณ์บางรายไม่สามารถสนับสนุนการเชื่อมต่อแบบซ็อกเก็ตในอุปกรณ์ MIDP ซึ่งหมายถึงว่าแอปพลิเคชันที่พัฒนาโดยใช้ซ็อกเก็ตอาจจะจำกัดเฉพาะในอุปกรณ์ไร้สายบางชนิด และเคลื่อนย้ายข้ามเครือข่ายอุปกรณ์ไร้สายต่างๆ ได้น้อยลง

ในการนำซ็อกเก็ตมาใช้ ก่อนอื่นผู้รับผู้ส่งจะต้องสร้างการเชื่อมต่อระหว่างซ็อกเก็ตเสียก่อน โดยฝ่ายหนึ่งจะคอยฟังคำร้องขอเชื่อมต่อ ขณะที่อีกฝ่ายหนึ่งขอเชื่อมต่อเครือข่าย เมื่อซ็อกเก็ตทั้ง 2 ด้านเชื่อมต่อเข้าด้วยกัน ก็จะส่งผ่านข้อมูลไปยังอีกด้านหนึ่ง

การรับข้อมูลจากเครื่องเซิร์ฟเวอร์ระยะไกลจะต้องสร้าง `InputConnection` ขึ้นมาเพื่อรับ `InputStream` จากการเชื่อมต่อ นั้น การส่งข้อมูลไปยังเครื่องเซิร์ฟเวอร์ระยะไกลจะต้องสร้าง `OutputConnection` ขึ้นมาเพื่อรับ `OutputStream` จากการเชื่อมต่อ นั้น ใน J2ME การเชื่อมต่อทั้ง 3 ประเภทที่กำหนดไว้เพื่อจัดการกับ Stream ข้อมูลขาเข้า / ขาออก ได้แก่ `InputStream` `OutputStream` และ `StreamConnection` โดยมีความหมายตามชื่อเรียก คือ `InputConnection` กำหนดให้ `input stream` สามารถรับส่งข้อมูลได้ ขณะที่ `OutputConnection` ก็กำหนดให้ `output stream` สามารถส่งข้อมูลได้ การจะนำการเชื่อมต่อประเภทใดมาใช้เมื่อไหร่ขึ้นอยู่กับการรับ – ส่ง หรือทั้งรับและส่งข้อมูล

การเขียนโปรแกรมแบบเครือข่ายโดยใช้ซ็อกเก็ตใน J2ME นั้นเขียนอย่างตรงไปตรงมา โดยมีขั้นตอนการทำงานดังนี้

1. เปิดการเชื่อมต่อแบบซ็อกเก็ตกับเซิร์ฟเวอร์ระยะไกลหรืออุปกรณ์ไร้สายอื่นโดยใช้ `Connector.open()`
2. สร้าง `InputStream` และ `OutputStream` จากเปิดการเชื่อมต่อแบบซ็อกเก็ตเพื่อใช้ในการรับ – ส่งกลุ่มข้อมูล
3. รับ – ส่งข้อมูลจากเซิร์ฟเวอร์ระยะไกลผ่านการเชื่อมต่อแบบซ็อกเก็ต ด้วยโอเปอเรชันการอ่านและเขียนในออบเจกต์ `InputStream` หรือ `OutputStream`
4. ปิดการเชื่อมต่อแบบซ็อกเก็ตและ `input / output stream` ก่อนออกจากโปรแกรม